

Runtime-Configurable Bit-Permutation Crossbar

Technical Design Document

*A 128-bit programmable bit-permutation engine
for low-latency syndrome routing in quantum error correction*

400 MHz

Fabric clock
(WNS +1.31 ns)

~83 ns

Reconfiguration
latency

4 cycles

Data-path
pipeline

Target platform: Xilinx Zynq UltraScale+ RFSoc (ZCU111)

Tooling: Vivado 2024.x | Language: SystemVerilog

March 2026

Executive Summary

This document describes the design, implementation and verification of a runtime-configurable 128-bit bit-permutation crossbar implemented in SystemVerilog on the Xilinx Zynq UltraScale+ RFSoc ZCU111 evaluation board. The component is a programmable bit-routing primitive for FPGA-based quantum error correction (QEC) pipelines: it applies an arbitrary user-defined permutation to a 128-bit syndrome word at line rate, and the permutation can be reconfigured at runtime in well under one microsecond — comfortably within a single QEC round on superconducting qubits.

The motivating application is real-time syndrome routing in quantum error correction (QEC). A QEC accelerator pipeline must apply a programmable bit-level permutation to a syndrome word at line rate, with the ability to reconfigure the permutation between rounds to support different qubit connectivity graphs and logical-qubit encodings without bitstream recompilation.

Key results:

- **Throughput:** one 128-bit word per clock cycle at 400 MHz, sustained.
- **Data-path latency:** 4 clock cycles (10 ns).
- **Reconfiguration latency:** ~83 ns total, comfortably under the 1 μ s budget.
- **Timing:** post-synthesis Worst Negative Slack of +1.31 ns at 400 MHz (standalone).
- **Resource:** 11% of LUTs and ~4% of flip-flops on the XCZU28DR device (integrated end-system, post-route).
- **Verification:** self-checking SystemVerilog testbench with 188 vectors across 7 test classes, plus end-to-end AXI4-Stream simulation through the full Aurora wrapper.
- **Integration:** AXI4-Stream wrapper enables drop-in integration into a complete Aurora 64B/66B receive-permute-transmit data path on the ZCU111.

The MUXF7/F8/F9 mux-tree architecture was selected after evaluating three alternatives (CARRY8 one-hot, Batcher bitonic sorter, Benes network). It wins on every metric that matters for the target use case: lowest data-path latency, simplest control generation, timing closure with positive margin at 400 MHz, and direct mapping onto dedicated UltraScale+ silicon primitives.

Contents

Executive Summary.....	2
1. Introduction.....	4
1.1 Background — Quantum Error Correction and Real-Time Decoding.....	4
1.2 Application — Programmable Syndrome Routing.....	4
1.4 References.....	5
1.5 Objectives.....	6
1.6 Document Structure.....	6
2. Platform Selection — Xilinx ZCU111.....	7
2.1 Key Requirements.....	7
2.2 ZCU111 Device Characteristics.....	7
2.3 Why ZCU111 Over Alternatives.....	8
3. System Architecture.....	9
3.1 End-to-End Data Path.....	9
3.2 AXI4-Stream Packet Protocol.....	9
4. Crossbar Design.....	11
4.1 Permutation Algebra.....	11
4.2 Memory Architecture.....	11
4.3 Data Pipeline.....	11
4.4 Configuration Protocol.....	12
5. AXI4-Stream Wrapper.....	13
6. Timing Analysis.....	14
6.1 Standalone Synthesis (400 MHz).....	14
6.2 Integrated Design Timing.....	14
6.3 Post-Implementation Routing Congestion.....	14
7. Verification.....	16
7.1 Crossbar Testbench.....	16
7.2 AXI-Stream Wrapper Testbench.....	16
7.3 Coverage Rationale.....	17
7.4 Formal Verification.....	17
8. Resource Utilisation.....	19
9. Key Design Decisions.....	20
10. Build and Simulation Instructions.....	21
11. Alternative Architectures Evaluated.....	24
12. Conclusions and Future Work.....	27
13. Implementation Findings — Achievable Permutation Width.....	28
Appendix A — Glossary.....	28

1. Introduction

1.1 Background — Quantum Error Correction and Real-Time Decoding

Quantum bits decohere. Even the best superconducting and trapped-ion qubits have physical error rates several orders of magnitude above what useful computation requires, and the rate at which errors accumulate forces any practical quantum computer to detect and correct them continuously. Quantum Error Correction (QEC) achieves this by encoding a single “logical” qubit into many physical qubits and repeatedly measuring a set of ancillary qubits whose outcomes — the *syndromes* — reveal where errors have occurred without disturbing the encoded information [1]. A classical *decoder* then consumes the syndrome stream and infers what correction to apply.

The decoder is the bottleneck of the system. On superconducting hardware a single round of syndrome extraction takes roughly one microsecond [2], and the decoder must keep up: if syndromes accumulate faster than they can be processed, the resulting *backlog problem* [3] forces exponential slowdown of the logical clock. Worse, certain fault-tolerant operations — most notably magic-state teleportation and feedback for non-Clifford gates — require the correction to land back at the qubits within a single QEC cycle. Real-time decoding latency therefore enters directly into the achievable logical clock rate of any error-corrected machine [2].

These constraints have driven QEC decoding onto FPGAs and custom silicon. Recent published results span a wide range of architectures and codes. Liyanage et al.’s distributed Union-Find decoder in the Helios architecture achieves an average decoding time of 11.5 ns per measurement round for a surface code of distance 21 on a single Xilinx VCU129 FPGA [4]. Their multi-FPGA extension, DecoNet, scales this to 100 logical qubits with lattice surgery across five interconnected FPGAs at an end-to-end average latency of 2.4 μ s [5]. Yang et al. recently demonstrated a hardware-integrated FPGA neural-network decoder for the distance-3 surface code, reporting a deterministic closed-loop latency of 550 ns (124 ns for the decoder itself) within a 1.25 μ s QEC cycle on a real superconducting processor [6]. Other open-source efforts — for instance the RISC-Q-based QEC system on AMD ZCU216 RFSocS — report sub-500 ns end-to-end decoding-feedback latencies for a distance-3 surface code [7].

Across all of these systems a common pattern emerges. The QEC pipeline is built from several FPGA-resident stages: a syndrome-extraction front end tied to the qubit-readout electronics, a routing and aggregation layer that funnels syndrome bits across one or more boards, the decoder itself, and a feedback path that returns Pauli-frame updates or conditional pulses to the control hardware. Each stage has its own latency budget, and the routing layer between them is what this project addresses.

1.2 Application — Programmable Syndrome Routing

Syndrome bits arrive at the decoder in an order dictated by the physical qubit layout and measurement circuitry: the readout multiplexers, the order of ancilla measurements within a stabiliser round, and the wiring of the cryogenic chain all contribute to a fixed hardware ordering. Decoders, however, expect their inputs in a logical order matched to the code being decoded — for the surface code, by stabiliser graph position; for the Gross code or other LDPC codes, by check-node index; for codes used in lattice-surgery experiments, by the boundary structure of the patch being merged. A bit-level permutation between the two orderings is therefore a standard primitive in any QEC FPGA pipeline.

In nearly all published implementations this permutation is hard-wired. The syndrome bus is routed at build time so that bit i of the raw measurement word lands on input j of the decoder, and changing the mapping requires re-synthesising the bitstream. This is acceptable for a fixed experiment but becomes a real obstacle when the same hardware platform must support:

- multiple qubit connectivity graphs (e.g. evaluating different defect-tolerant routings on the same physical chip),
- multiple QEC codes (surface code at varying distances, colour codes, LDPC codes, the Gross code),
- multi-patch lattice-surgery operations where the boundary mapping changes between rounds, or
- multi-tenant systems where different users target different logical-qubit encodings on shared FPGAs.

The component described in this document addresses exactly this gap. It is a runtime-configurable bit-permutation crossbar (128 bits in the integrated ZCU111 build): at line rate it applies an arbitrary permutation P to a 128-bit syndrome word ($\text{data_out}[i] = \text{data_in}[P[i]]$), and the permutation can be reprogrammed over a 64-bit configuration bus in well under one microsecond — comfortably within a single QEC round on superconducting qubits. The same bitstream can therefore serve many different qubit layouts, codes, and decoder topologies without recompilation, and the routing layer ceases to be a build-time constant.

The target platform is the Xilinx Zynq UltraScale+ RFSoc ZCU111 evaluation board. The RFSoc family is now an established platform in quantum control hardware: the open-source QICK qubit controller [8], for example, is built on the same device, leveraging the integrated 4 GSPS ADCs and 6.5 GSPS DACs for direct synthesis of qubit pulses below 6 GHz. Choosing the ZCU111 places the design firmly within an existing ecosystem of quantum-control firmware and reference designs.

1.3 The Crossbar in a Quantum-Computing System

Sections 1.1 and 1.2 establish why a programmable syndrome-routing layer is needed. This section places the crossbar concretely within the end-to-end

operation of an error-corrected quantum computer, describing the stages that produce its input and the stages that consume its output. The crossbar is one block in a closed real-time loop that runs continuously for as long as a logical qubit is kept alive; understanding the blocks on either side is necessary to understand the latency and interface constraints the design must meet.

1.3.1 Upstream — From Physical Qubits to a Syndrome Word

The chain that produces the crossbar's input begins at the physical qubits. A logical qubit is encoded across many physical qubits using a quantum error-correcting code, most commonly the surface code, in which data qubits hold the encoded state and ancilla qubits are used to measure parity checks. At a fixed cadence — one stabiliser-measurement round, roughly one microsecond on current superconducting hardware — the control system executes a short circuit that entangles each ancilla with its neighbouring data qubits and then measures the ancilla. These measurements are deliberately constructed so that they reveal only parity information and do not collapse the encoded logical state.

Each ancilla measurement is a physical, analogue event. For superconducting qubits it is a microwave readout pulse whose amplitude and phase must be digitised and classified. On an RFSoc platform such as the ZCU111 this is exactly the role of the integrated RF data converters and the fabric logic around them: the ADC samples the readout tone, and a discrimination stage reduces each measurement to a single classical bit. The raw per-round bit pattern is then differenced against the previous round — a stabiliser whose value has changed signals that an error occurred nearby. The resulting sparse binary vector of changed checks is the syndrome, and it is the data word presented to the permutation crossbar. Upstream of the crossbar, in order, the stages are therefore: stabiliser measurement, analogue readout digitisation, binary discrimination, and syndrome differencing.

Two characteristics of this upstream chain shape the crossbar's interface. First, the syndrome is sparse: in a well-behaved device most stabilisers do not change between rounds, so the data word is mostly zeros with a small number of set bits marking detection events. The crossbar must nonetheless route every bit position, because which positions are set is not known in advance and changes every round. Second, the bit ordering at the readout stage is rigid. It is fixed by three things that are decided at hardware-design time: the physical placement of qubits on the chip, the assignment of readout resonators to frequency-multiplexed ADC channels, and the scheduling order in which ancillae are measured within a round. None of these is chosen for the convenience of the decoder, and none can be changed without modifying the cryogenic hardware or the readout firmware.

It is worth being precise about what a single syndrome word contains, because this determines how many permutation elements the crossbar needs. In the simplest scheme one element carries one stabiliser's differenced outcome for one round. A distance- d surface code has of order $d^2 - 1$ stabilisers, so a single-round syndrome for a distance-11 code is approximately

120 bits — within the 128-element width of this design. Schemes that decode over a sliding window of several rounds, or that attach soft information such as a measurement-confidence value to each event, multiply this count and are the situations in which a wider permutation, and therefore the scaling path of Section 13.6, becomes necessary.

1.3.2 The Crossbar — Reordering the Syndrome for the Decoder

The syndrome bits leave the readout stage in an order fixed by the hardware: the physical qubit layout, the readout-multiplexer assignment, and the order in which ancillae are measured within a round. A decoder, however, expects its inputs in a logical order matched to the structure of the code it is decoding — for the surface code, by position in the stabiliser graph; for an LDPC code, by check-node index. The crossbar applies the permutation that maps the hardware ordering onto the decoder ordering. Because it is runtime-configurable, the same bitstream can serve different codes, different qubit-connectivity graphs, and different logical-qubit encodings: only the two index maps loaded over the configuration bus change, not the hardware. This is the single function of the block this document describes, and it sits directly between the syndrome-differencing stage and the decoder.

The reordering the crossbar performs is best understood as a change of basis between two indexing schemes for the same set of parity checks. The readout stage emits check k of the hardware ordering on a fixed wire; the decoder expects check k of its graph ordering on a fixed input. The permutation P is exactly the bijection between these two labellings. Computing P is not the crossbar's job — it is produced offline from the code definition and the hardware floorplan, and delivered to the crossbar as two index maps over the configuration bus. The crossbar's job is only to apply P at line rate, every round, for as long as the logical qubit is live.

Placing this function in reconfigurable hardware, rather than hard-wiring it, is a deliberate architectural choice. A fixed permutation could be implemented as static routing and would cost almost nothing, but it would bind one bitstream to exactly one code, one qubit-connectivity graph, and one readout-channel assignment. In a research or multi-tenant QEC system these change frequently: a different code distance, a relabelled patch after a lattice-surgery operation, or a remapped readout multiplexer all alter the required permutation. A runtime-configurable crossbar absorbs all of these by reloading two index maps in roughly 83 nanoseconds, with no place-and-route and no bitstream regeneration. The cost of that flexibility is the configuration logic, the composition step, and the mux fabric analysed in the rest of this document.

1.3.3 Downstream — From Permuted Syndrome to Correction

The permuted syndrome feeds the decoder, which is the computationally hard stage of the loop and the one that dominates the latency budget. The decoder infers, from the pattern of changed checks, the most likely set of underlying physical errors; common algorithms are minimum-weight perfect matching,

union-find, and belief propagation, and recent systems also use FPGA-resident neural-network decoders. Its output is a correction. In most architectures the correction is not applied as a physical operation on the qubits but is tracked in software as an update to the Pauli frame — a running record of accumulated corrections that is folded into the interpretation of later measurements. The exception is feedback for non-Clifford operations, such as magic-state teleportation, where the correction must return to the control hardware and condition the next pulses within the same QEC cycle. Downstream of the crossbar, in order, the stages are therefore: the decoder, and then either Pauli-frame tracking or a hardware-feedback path back to qubit control.

The asymmetry between the crossbar and the decoder is the central reason the crossbar can be made simple. Decoding is, in the general case, computationally hard: minimum-weight perfect matching on the surface-code graph is polynomial but not cheap, union-find trades accuracy for speed, and belief propagation may not converge for every syndrome. Real-time decoders are the subject of active research precisely because keeping their latency bounded under a continuous syndrome stream is difficult. The permutation, by contrast, is a fixed-cost operation: it is the same amount of work regardless of the syndrome content, it cannot fail to converge, and its latency is a constant known at design time. Spending fabric and effort to make the permutation stage cheap and predictable is therefore the correct division of resources.

The distinction between Pauli-frame tracking and hardware feedback also has a direct bearing on the crossbar's requirements. For Clifford operations the correction need not be applied physically; it is accumulated in the Pauli frame, a software record, and the real-time pressure on the loop is throughput rather than latency. For non-Clifford operations — magic-state injection and teleportation — the decoder result must return to the control electronics in time to condition the next physical pulses, and the whole loop, crossbar included, is then on the critical latency path. The crossbar is designed for the stricter of the two cases: its latency is a small, fixed, documented constant so that it never contributes meaningfully to a feedback deadline.

1.3.4 Why the Crossbar's Latency Budget Is What It Is

The 1 microsecond figure that recurs throughout this document is the stabiliser-round period: the rate at which fresh syndromes are produced. The real-time constraint on the loop has two parts. First, throughput — the routing and decoding pipeline must on average consume syndromes at least as fast as they arrive, or the well-known backlog problem forces the logical clock to slow down. Second, for feedback-dependent operations, latency — the correction must return within a bounded number of rounds. Within this budget the decoder is the expensive stage; the permutation crossbar is deliberately cheap, contributing only its four-cycle data-path latency (10 ns at 400 MHz) and a sub-microsecond, in fact roughly 83 ns, reconfiguration time when the permutation must change. The crossbar is designed so that the routing layer is never the bottleneck in the loop: it adds negligible latency, leaves the bulk of the FPGA fabric free for

decoder accelerators, and can be reprogrammed between rounds when the code or qubit layout changes. Its place in the system is a small, fast, reconfigurable connective stage between a fixed-by-hardware syndrome source and a logical-order-expecting decoder.

1.3.5 Integration Constraints the System Context Imposes

Viewing the crossbar as one block in this loop makes its non-obvious interface requirements concrete. Because it sits in a continuously running pipeline, it must accept a new syndrome every round without gaps: the AXI4-Stream handshake and the skid-buffered I/O pipelines described in Section 5 exist so that upstream and downstream stalls never corrupt or drop a syndrome. Because the permutation may change between logical operations but not within a round, configuration and data use the same stream but distinct phases — the two index maps are loaded first, the data word follows — and the design forbids a configuration change from taking effect mid-word. Because the crossbar shares the FPGA with the readout data converters, the decoder accelerators, and the processing system, it must be modest in both logic and routing footprint; this is the system-level reason the integrated design is held to a width that places and routes comfortably alongside everything else, rather than the largest width that fits in isolation.

Finally, the loop context explains why the crossbar is delivered as a self-contained, parameterised IP with a streaming interface rather than as bespoke logic fused into a particular decoder. The syndrome-routing layer is the part of a QEC accelerator most likely to be reused unchanged across experiments, codes, and even decoder algorithms, since its function — apply a relabelling — is independent of how the decoder works. Packaging it as an independent block with a clean AXI-Stream interface lets it be dropped into different QEC pipelines, and lets the decoder behind it be replaced, without touching the routing layer at all.

1.3.6 The End-to-End System: A Worked Example

It is useful to trace a single quantum error-correction cycle through the complete system in concrete terms, so that the crossbar's role and timing are unambiguous. Consider an error-corrected quantum computer holding one logical qubit in a distance-7 surface code on a superconducting processor — the configuration in which Google Quantum AI first demonstrated a quantum memory operating below the error-correction threshold [9], a 101-physical-qubit patch with roughly 48 stabilisers. The cycle proceeds as follows.

First, the control electronics execute the stabiliser-measurement circuit: each ancilla qubit is entangled with its neighbouring data qubits and then read out. The readout is an analogue microwave event. On an RFSoc platform such as the ZCU111, the integrated data converters digitise the readout tone and fabric logic discriminates each ancilla into one classical bit. Differencing this bit pattern against the previous cycle produces the syndrome — the sparse binary vector of stabilisers whose value changed. For a distance-7 patch this is on the order of 48

bits, comfortably inside the 128-element width of the crossbar described in this document. This entire front-end stage must complete within the stabiliser-round period, approximately one microsecond on current superconducting hardware.

Second, the syndrome enters the crossbar. It arrives in the order fixed by the hardware — qubit floorplan, readout-multiplexer assignment, ancilla measurement schedule — and the crossbar applies the permutation that reorders it into the decoder's expected graph ordering. With the two index maps already loaded, this costs only the four-cycle data-path latency of the design, about 10 ns at 400 MHz. If the experiment switches to a different code or a relabelled patch — for example after a lattice-surgery operation — the host reloads the two maps in roughly 83 ns, with no bitstream regeneration.

Third, the permuted syndrome reaches the decoder, the computationally hard stage. Recent FPGA-resident decoders illustrate the budget: a Collision Clustering decoder sustains megahertz throughput on surface-code patches of several hundred qubits [2]; an FPGA neural-network decoder has demonstrated a 124 ns decoding latency inside a 550 ns closed-loop demonstration on a distance-3 code [6]; and a fully integrated open-source RFSoc system has reported an end-to-end decoding-feedback latency of 446 ns for a distance-3 surface code, including syndrome aggregation, inter-board communication, decoding, and correction distribution [7]. In every one of these systems the decoder, the communication, and the feedback path dominate the latency budget; the syndrome-routing permutation does not. The decoder emits a correction, which is either folded into the software Pauli frame or, for non-Clifford operations, returned to the control hardware to condition the next pulses.

This worked example makes the design rationale explicit. The crossbar sits on the real-time path of a loop whose deadline — one stabiliser round, of order one microsecond — is set by the physics of the qubits, not by the FPGA. The loop stays stable only if every stage keeps pace; a stage that falls behind causes the well-known backlog problem, in which undecoded syndromes accumulate and force the logical clock to slow exponentially [2]. The crossbar is therefore engineered to be the cheapest stage in the loop: a small, fixed, predictable latency, a modest fabric footprint that leaves room for the decoder accelerators, and runtime reconfigurability so that the same bitstream serves whatever code, qubit layout, or decoder the experiment requires. Demonstrations of real-time QEC on superconducting hardware [10] confirm that this sub-microsecond budget is now routinely met in practice, and that the connective stages around the decoder must be correspondingly lightweight — which is precisely the niche this design fills.

1.3.7 The Full System Stack: Where the Crossbar Sits

The preceding subsections followed the syndrome signal along its path. It is equally useful to view the same system top-down, as a stack of abstraction layers, because this is how error-corrected quantum computers are conventionally designed and analysed [11]. Each layer presents a service to the layer above and is built on the layer below, and the layers operate on time scales

that differ by several orders of magnitude — from nanosecond physical pulses to millisecond logical gates. The crossbar described in this document is a small but load-bearing component of one specific layer, and locating it in the full stack clarifies both what it must do and what it must not be asked to do.

A practical superconducting quantum computer can be described in four layers. The physical layer is the quantum processor itself: the data and ancilla qubits, held at millikelvin temperature, on which the encoded logical state lives. The control and readout layer is the classical hardware that drives the physical layer — the arbitrary-waveform generators that synthesise gate pulses, and the readout chain that digitises and discriminates ancilla measurements; on an RFSoc platform such as the ZCU111 this layer is largely the integrated data converters and the surrounding programmable logic [8]. The classical processing layer is where quantum error correction is actually computed: syndrome differencing, the permutation performed by this crossbar, the decoder, and the correction logic. The logical layer is the topmost: it sees only clean, error-corrected logical qubits and logical gates, and is where lattice surgery, magic-state distillation, and ultimately the quantum algorithm are expressed. The crossbar lives entirely inside the classical processing layer, on the boundary between the control-and-readout layer that produces raw syndromes and the decoder that consumes them.

The table below summarises the four layers, their characteristic time scales, and the place of the crossbar within them.

Layer	Function	Time scale	Crossbar relevance
Logical	Error-corrected logical qubits and gates; lattice surgery; algorithm execution	Microseconds to milliseconds per logical gate	Above the crossbar; unaware of it. A logical operation may trigger a permutation change.
Classical processing	Syndrome differencing, permutation routing, decoding, correction tracking	$\sim 1 \mu\text{s}$ per QEC round	The crossbar is here, between syndrome differencing and the decoder.
Control and readout	Pulse generation; readout digitisation and discrimination	Nanoseconds per pulse; $\sim 1 \mu\text{s}$ per readout	Produces the raw syndrome word in hardware order; defines the input permutation.
Physical	Data and ancilla qubits in a QEC code at millikelvin	$\sim 10\text{--}100$ ns per physical	Below the crossbar; the qubit floorplan ultimately fixes the

		gate	permutation.
--	--	------	--------------

Two cross-layer points complete the picture. First, the layered view explains why the permutation belongs in the classical processing layer and not, say, baked into the control-and-readout firmware: the routing must change when the logical layer above changes the code or relabels a patch, so it has to be reconfigurable from software, yet it must run at the round rate of the layer below, so it has to be hardware. A small runtime-configurable hardware block in the classical processing layer is exactly the structure that satisfies both. Second, the classical processing layer as a whole is increasingly recognised as a first-class part of the machine rather than a peripheral: end-to-end designs now integrate decoding firmware directly with the control processor so that syndrome routing, decoding, and feedback all meet the one-microsecond round budget together [12]. The crossbar is one component engineered to that standard — it occupies a clearly bounded role in a single layer, exposes a clean streaming interface to the stages on either side of it, and is deliberately lightweight so that the decoder, the hardest tenant of the same layer, retains the fabric and the time budget it needs.

1.4 References

- [1] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Phys. Rev. A*, vol. 86, no. 3, p. 032324, Sep. 2012.
- [2] B. Barber et al., “A real-time, scalable, fast and resource-efficient decoder for a quantum computer,” *Nature Electronics*, vol. 8, pp. 84–91, 2025.
- [3] B. M. Terhal, “Quantum error correction for quantum memories,” *Rev. Mod. Phys.*, vol. 87, no. 2, p. 307, 2015.
- [4] N. Liyanage, Y. Wu, A. Deters, and L. Zhong, “FPGA-based Distributed Union-Find Decoder for Surface Codes,” *IEEE Trans. Quantum Engineering*, vol. 5, 2024. arXiv:2406.08491.
- [5] N. Liyanage et al., “DecoNet: A Network-Integrated Decoding System for Real-Time Quantum Error Correction with Lattice Surgery,” arXiv:2504.11805, 2025.
- [6] X. Yang et al., “Real-time Surface-Code Error Correction Using an FPGA-based Neural-Network Decoder,” arXiv:2605.04892, 2026.
- [7] Y. Wu et al., “A Scalable Open-Source QEC System with Sub-Microsecond Decoding-Feedback Latency,” arXiv:2603.16203, 2026.
- [8] L. Stefanazzi et al., “The QICK (Quantum Instrumentation Control Kit): Readout and control for qubits and detectors,” *Rev. Sci. Instrum.*, vol. 93, p. 044709, 2022. arXiv:2110.00557.
- [9] Google Quantum AI and Collaborators, “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, pp. 920–926, 2025. arXiv:2408.13687.

- [10] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” arXiv:2410.05202, 2024.
- [11] N. C. Jones, R. Van Meter, A. G. Fowler, P. L. McMahon, J. Kim, T. D. Ladd, and Y. Yamamoto, “Layered architecture for quantum computing,” *Phys. Rev. X*, vol. 2, no. 3, p. 031007, Jul. 2012.
- [12] F. Zhang et al., “A classical architecture for digital quantum computers,” *ACM Trans. Quantum Computing*, 2024. arXiv:2305.14304.

1.5 Objectives

The specific engineering objectives for this design were:

- **Functional:** implement a programmable permutation that computes $\text{data_out}[i] = \text{data_in}[P[i]]$ for a user-defined permutation vector P , sustaining one permutation word per clock cycle.
- **Reconfiguration:** support runtime reconfiguration of the permutation over a 64-bit configuration bus, with total reconfiguration latency under 1 microsecond.
- **Timing:** close timing at 400 MHz on a Xilinx UltraScale+ RFSoc device.
- **Integration:** wrap the crossbar in an AXI4-Stream interface to enable seamless integration with an Aurora 64B/66B serial link, forming a complete end-to-end receive-permute-transmit data path on the ZCU111.
- **Verification:** demonstrate functional correctness through a self-checking simulation framework covering identity, reversal, random, back-to-back reconfiguration, and broadcast test cases.

1.6 Document Structure

Section 2 explains the choice of platform. Section 3 describes the overall system architecture and the AXI4-Stream packet protocol used between Aurora and the crossbar. Section 4 covers the crossbar internals: the permutation algebra, memory architecture, data pipeline, and configuration protocol. Section 5 describes the AXI4-Stream wrapper. Section 6 reports timing analysis. Section 7 documents the verification framework and proposed formal-verification plan. Section 8 reports resource utilisation. Section 9 records the key design decisions. Section 10 covers the build flow. Section 11 documents the alternative architectures that were evaluated and rejected. Section 12 concludes and outlines future work.

2. Platform Selection — Xilinx ZCU111

The Xilinx Zynq UltraScale+ RFSoc ZCU111 evaluation board was selected as the target platform for this design. This section documents the rationale for that selection against the requirements of the QEC data-routing use case.

2.1 Key Requirements

- High-speed serial connectivity to transmit and receive syndrome data between FPGA nodes at multi-Gbps rates, with deterministic latency.
- Sufficient programmable-logic fabric to host both the permutation crossbar and one or more decoder accelerators simultaneously.
- A tightly integrated processing system for configuration, monitoring and control without requiring a separate host CPU board.
- A well-supported, production-grade device with Vivado toolchain support, an IP ecosystem (Aurora, AXI interconnect), and a long-term availability commitment from the vendor.

2.2 ZCU111 Device Characteristics

Feature	ZCU111 / XCZU28DR-2e	Relevance to this design
Programmable Logic	425,280 LUTs, 850,560 FFs, 2,160 BRAMs	Sufficient for crossbar (11% LUTs, integrated) and decoder accelerators in remaining fabric
GTY Transceivers	12 × 33 Gbps GTY on FMC+; SFP28 cage for 4 modules	Direct support for Aurora 64B/66B at 25.78 Gbps over SFP28 without additional hardware
Processing System	Quad-core Arm Cortex-A53 + dual Cortex-R5	PS provides pl_clk0 (400 MHz fabric clock), AXI-GPIO for Aurora reset, and Linux for control software
RF Data Converters	8× 12-bit 4 GSPS ADCs, 8× 14-bit 6.5 GSPS DACs	Not used here, but relevant for future integration with qubit readout electronics
SD-FEC Cores	8 hardened soft-decision FEC blocks	Potential acceleration of LDPC/turbo decoding in future QEC decoder iterations
Memory	4 GB DDR4 on PL, 4 GB DDR4 SODIMM on PS	Available for decoder graph storage and syndrome

Feature	ZCU111 / XCZU28DR-2e	Relevance to this design
		buffering
Speed grade	-2e (Extended, enhanced timing)	Supports 400 MHz fabric clock with positive timing margin

2.3 Why ZCU111 Over Alternatives

The primary driver for selecting the ZCU111 over other UltraScale+ boards (e.g. ZCU102, VCU118) is the combination of on-board SFP28 cages and GTY transceivers that directly support 25 Gbps Aurora links without an FMC daughtercard. In a multi-FPGA QEC system each node must communicate with neighbouring decoder nodes at high bandwidth and low latency; having the transceiver hardware directly on the board eliminates a connector layer and simplifies the mechanical design.

The RFSoc device family is also increasingly prevalent in quantum control stacks. The QICK (Quantum Instrumentation Control Kit) project, developed for superconducting qubit experiments, uses the ZCU111 as its primary hardware platform, leveraging the integrated RF-DACs and RF-ADCs for direct qubit pulse generation and readout at frequencies up to 6 GHz. This establishes the ZCU111 as a known-good platform in the quantum hardware ecosystem, with an active community, open-source reference designs, and validated integration with cryogenic measurement setups.

The Zynq PS is important for this design specifically because it provides a deterministic 400 MHz pl_clk0 to the programmable logic, drives the Aurora reset via AXI-GPIO, and can run a Linux-based control stack for configuration management — all without requiring a separate host computer. The Cortex-R5 real-time processor is available for hard real-time control tasks with guaranteed response latency, which is directly relevant for the QEC feedback loop.

3. System Architecture

3.1 End-to-End Data Path

The complete system deployed on the ZCU111 consists of the following components, connected as an AXI4-Stream pipeline:

Block	Type	Function
Aurora 64B/66B	Xilinx IP	25.78 Gbps GTY serial link; receives/transmits 64-bit AXI-S beats at ~391 MHz (user_clk_out)
axis_data_fifo_0	Xilinx IP	Async FIFO; bridges Aurora user_clk_out domain to the crossbar clock domain
crossbar_axis_wrapper	Custom RTL	AXI-S slave/master wrapper; sequences configuration and data over the crossbar
runtime_configurable_crossbar	Custom RTL	Programmable permutation engine, 128 bits in this build (this design)
axis_data_fifo_1	Xilinx IP	Async FIFO; bridges crossbar output back to Aurora transmit clock domain
Zynq UltraScale+ PS	Xilinx IP	Provides pl_clk0 (400 MHz) and AXI-GPIO for Aurora reset control

The PS provides the 400 MHz fabric clock (pl_clk0) that clocks the crossbar and its wrapper. The Aurora core operates on its own user_clk_out (~391 MHz); the two async FIFOs perform the clock-domain crossing at ingress and egress.

3.2 AXI4-Stream Packet Protocol

The RX AXI-Stream packet from Aurora carries three concatenated payloads in a single 32-beat transaction (for the 128-bit build):

Beat range	Beats	Content	cfg_sel
0..14	15	orig_map: 9 × 7-bit indices per beat, defining the original bit ordering	0
15..29	15	final_map: 9 × 7-bit indices per beat, defining the desired output ordering	1
30..31	16	Input bitstring: 2 × 64-bit beats assembling the 128-bit data word	N/A

tlast is asserted only on beat 31 (the final data beat). Phase transitions are driven purely by beat count so that the receiver does not rely on tlast for configuration framing, making the protocol robust to upstream framing errors.

The TX AXI-Stream response is 2 beats of 64-bit data (the permuted 128-bit word) with tlast on beat 1.

4. Crossbar Design

4.1 Permutation Algebra

The user supplies two index vectors over the configuration bus. The crossbar computes their composition to derive the final permutation applied to the data path:

```
inv_orig[orig_map[i]] = i           // invert the original mapping
P[i] = inv_orig[final_map[i]]      // compose with final mapping
data_out[i] = data_in[P[i]]        // apply permutation
```

This two-map representation allows the upstream system to express the permutation as a change-of-basis between an original labelling and a desired output labelling, without requiring the sender to pre-compute the composition. The hardware performs the inversion and composition once at configuration time; the data path then applies only the pre-computed vector P.

4.2 Memory Architecture

Three storage structures implement the configuration and data path:

Array	Implementat ion	Size	Write address	Purpose
inv_orig_reg	LUTRAM (distributed)	128 × 7b	Data- dependent (cfg_data value)	Inverse of orig_map; built during cfg_sel=0 loading
final_req_reg	LUTRAM (distributed)	128 × 7b	Sequential counter	Stores final_map verbatim; built during cfg_sel=1 loading
src_for_out	Registers	128 × 7b	Sequential counter (composition)	Pre-computed permutation P; read in parallel by mux tree

LUTRAM is chosen for inv_orig_reg and final_req_reg because both require asynchronous parallel read during the composition step — BRAM cannot support 128 simultaneous reads. src_for_out is implemented as plain registers since it must be read by all 128 mux instances simultaneously.

4.3 Data Pipeline

The data path is a 4-cycle pipeline from data_in_vld to data_out_vld:

Cycle	Stage	Logic
1	Broadcast register tree	data_in replicated into independent register banks (clusters). Prevents the 128-bit net from becoming a routing bottleneck and allows Vivado to place each cluster near its mux group.
2	Mux stage A (in mux_f789_tree)	Each cluster drives 32 independent mux_f789_tree instances. Stage A: 16:1 selection using select[3:0], implemented as LUT6+MUXF7+MUXF8 per output bit. Output registered.
3	Mux stage B (in mux_f789_tree)	64:1 selection using select[9:4], implemented as a MUXF7/F8/F9 chain. Output registered.
4	Output register	mux_raw_out registered to data_out; vld_pipe[2] asserts data_out_vld.

The mux_f789_tree submodule explicitly targets Xilinx UltraScale+ slice primitives (MUXF7, MUXF8) to implement the 128:1 selection in the minimum number of logic levels. The (* dont_touch *) attribute prevents Vivado from merging pipeline registers across stage boundaries.

4.4 Configuration Protocol

Configuration proceeds in two phases, back-pressured via cfg_ready:

- **Phase 1 (cfg_sel=0):** 15 beats load orig_map. Each 64-bit beat carries 9×7 -bit indices. The beat counter provides the write value; the index value in cfg_data provides the write address (building the inverse map directly).
- **Phase 2 (cfg_sel=1):** 15 beats load final_map. Sequential write address (counter) enables BRAM-style inference.
- **Composition:** once both done_a and done_b assert, a single resolving pulse triggers $\text{src_for_out}[j] \leftarrow \text{inv_orig_reg}[\text{final_req_reg}[j]]$ for all 128 j simultaneously.
- **Ready:** ready_out asserts two cycles after resolving, guaranteeing the composition write has settled before data_in_vld is accepted.

Total reconfiguration latency at 400 MHz:

Phase	Cycles	Time
Load orig_map (15 beats)	15	37.5 ns
Load final_map (15 beats)	15	37.5 ns
Composition + ready_out	~3	7.5 ns

Phase	Cycles	Time
Total	~33	~82.5 ns

This is comfortably under the 1 microsecond budget, leaving approximately 137 ns of margin.

5. AXI4-Stream Wrapper

5.1 Overview

The `crossbar_axis_wrapper` module presents a standard AXI4-Stream slave/master interface to the rest of the system, hiding the crossbar's bespoke configuration bus. It implements two independent state machines: an RX FSM that sequences the three configuration and data phases, and a TX FSM that serialises the permuted output word back to 64-bit beats.

5.2 RX State Machine

State	Action	Transition
RX_CFG_A	Drive <code>cfg_sel=0</code> , accept beats, forward to crossbar	After <code>BEATS_CFG=15</code> accepted beats → RX_CFG_B
RX_CFG_B	Drive <code>cfg_sel=1</code> , accept beats, forward to crossbar	After <code>BEATS_CFG=15</code> accepted beats → RX_WAIT
RX_WAIT	Deassert <code>tready</code> (hold upstream); wait for crossbar <code>ready_out</code>	<code>ready_out</code> asserts → RX_DATA
RX_DATA	Assemble 2×64 -bit beats into 128-bit shift register; assert <code>data_in_vld</code> on last beat	After <code>BEATS_DATA=2</code> beats, <code>tlast</code> present → RX_IDLE
RX_IDLE	One-shot complete	Stays until reset
RX_ERROR	Sticky framing error (<code>tlast</code> absent on final beat)	Stays until reset

`tready` is deasserted during RX_WAIT so that the Aurora FIFO buffers any incoming beats while the crossbar completes its ~ 3 -cycle composition. This ensures no beats are dropped and the packet framing is preserved.

5.3 TX State Machine

On `data_out_vld` assertion, the TX FSM captures the 128-bit `data_out` word into a shift register and serialises it as 2 consecutive 64-bit AXI-Stream beats, asserting `tlast` on beat 1. The TX path is back-pressured by `m_axis_tx_tready` so it handles downstream stalls correctly.

6. Timing Analysis

6.1 Standalone Synthesis (400 MHz)

Post-synthesis timing report for the standalone crossbar (DATA_WIDTH=128, xczu28dr-2e):

Metric	Value
Clock period	2.500 ns (400 MHz)
Worst Negative Slack (WNS)	+1.311 ns
Worst path	done_b_reg → final_req_reg[38][*]/CE, 3 LUT levels, 0.948 ns
Status	TIMING MET

6.2 Integrated Design Timing

When integrated in the full Aurora block design, the crossbar is clocked by pl_clk0 from the Zynq PS, which drives a clock tree with a fanout of over 1.3 million. This creates a source-clock delay of approximately 2.9 ns before any logic runs, consuming most of the 2.5 ns clock period budget on the clock path alone.

Three configuration-phase paths were identified as timing-critical in this context. All three are write paths that are only active during reconfiguration (never during data processing), so multicycle path constraints are applied safely:

Path	Constraint	Rationale
count_b → final_req_reg[*] [*]/CE	set_multicycle_path -setup 2	Sequential write address decode; 10 LUT levels through CARRY8 comparators; config-only
cfg_data → inv_orig_reg[*] [*]	set_multicycle_path -setup 2	Data-dependent write address; large fanout decode; config-only
resolving → src_for_out[*]	set_multicycle_path -setup 2	Double LUTRAM read (inv_orig + final_req); config-only

Each multicycle path constraint is paired with a corresponding -hold 1 constraint to ensure hold time is correctly tightened. Two separate constraint files are provided: constraints.xdc uses full hierarchical cell paths for the integrated

Aurora block design, and `constraints_standalone.xdc` uses unqualified cell names for standalone crossbar implementation.

6.3 Post-Implementation Routing Congestion

Place-and-route of the integrated 128-bit design completes cleanly: the design is fully routed with zero routing errors, and post-route utilisation is approximately 11% of LUTs (see Section 8). At 128 bits the crossbar mux tree is not a routing bottleneck. Routing congestion does, however, become the binding constraint at larger permutation widths: a 256-bit flat crossbar is unroutable in the integrated system, and the 1024-bit configuration does not place at all. The full width-versus-routability analysis, including the measured congestion levels and the mitigations attempted, is given in Section 13.

Three implementation directives are applied in `prj.tcl` to manage routing congestion. They are sufficient at 128 bits and were also evaluated, without success, as mitigations for the 256-bit configuration (see Section 13.3):

Directive	Tcl property	Effect
Spread placement	<code>STEPS.PLACE_DESIGN.ARG S.DIRECTIVE = AltSpreadLogic_high</code>	Distributes the mux tree cells evenly across the fabric, reducing hotspots in the central CLB region
Aggressive routing	<code>STEPS.ROUTE_DESIGN.ARG S.DIRECTIVE = AggressiveExplore</code>	Enables more rip-up/reroute iterations and a wider search for detour paths when blocked resources are encountered
Pblock floorplan	<code>create_pblock + resize_pblock SLICE_X40Y0:SLICE_X79Y299</code>	Confines crossbar logic to the right half of the fabric, preventing interleaving with the AXI/Aurora wrapper logic which is the primary driver of routing detours

The pblock boundary can be adjusted if the design is instantiated in a larger hierarchy — shift the SLICE range right if the congestion hotspot reported by Vivado falls within the pblock. For standalone implementation the pblock can be removed since there is no competing logic.

Note that the `-hold 1` multicycle constraint is silently ignored during synthesis (Vivado warns Designutils 20-1567) but is correctly applied during implementation. This is expected behaviour — hold constraints are implementation-only.

7. Verification

7.1 Crossbar Testbench (`tb_runtime_configurable_crossbar`)

A self-checking SystemVerilog testbench verifies the crossbar in isolation at 400 MHz. The testbench implements the same permutation algebra as the RTL in software (a golden model) and compares every output word bit-for-bit against the expected result. The configuration-bus protocol is exercised through a back-pressured task that respects `cfg_ready`.

Test case	Description	Vectors
TC-01: Identity	<code>orig_map = final_map = [0..1023]</code> . <code>P[i]=i</code> so <code>data_out == data_in</code> . Verifies basic connectivity and pipeline latency.	20
TC-02: Bit reversal	<code>final_map[i] = 1023-i</code> . Verifies correct handling of a fully reversed permutation.	20
TC-03: Random permutation	Both maps independently shuffled (Fisher-Yates). No input gaps. Verifies correct composition for an arbitrary permutation.	30
TC-04: Random + input gaps	Random permutation with ~20% stall cycles on <code>data_in_vld</code> . Verifies pipeline stall handling and scoreboard correctness.	50
TC-05: Back-to-back reconfig	Two sequential reconfigurations without full reset. Verifies <code>cfg_restart</code> correctly clears previous state.	2×10
TC-06a: Broadcast	All outputs map to <code>input[42]</code> . Non-permutation pattern; verifies degenerate mapping handling.	20
TC-06b: Dualcast	Even outputs from <code>input[0]</code> , odd from <code>input[1023]</code> . Verifies multi-source fan-out.	20

The testbench also measures wall-clock configuration time from `cfg_restart` to `ready_out` for every test case and reports a LATENCY VIOLATION error if the 1 microsecond budget is exceeded.

7.2 AXI-Stream Wrapper Testbench (`tb_crossbar_axis_wrapper`)

A separate testbench exercises the complete AXI4-Stream interface at 400 MHz. A pre-computed 32-beat RX packet (15 beats of `orig_map`, 15 beats of `final_map`,

2 beats of data) is streamed back-to-back with `tvalid` held high throughout. The testbench measures the cycle count from the first accepted RX beat to TX `tlast` acceptance and verifies:

- The 128-bit output word matches the identity-permuted input (`data_out == data_in` for identity maps).
- Total end-to-end latency: 15 (CFG_A) + 15 (CFG_B) + ~2 (composition) + 2 (data in) + 4 (crossbar pipeline) + 2 (TX out) = ~40 cycles $\approx$ 100 ns, within the 1 μ s budget.
- `rx_framing_err` is not asserted.
- A global watchdog fires after 10,000 cycles to catch any hang condition.

7.3 Coverage Rationale

The test suite targets the following coverage goals:

- **Permutation correctness:** random shuffles (TC-03/04/05) provide statistical confidence across the full 1024! permutation space. The Fisher-Yates shuffle generates uniformly random permutations, exercising the full address space of both LUTRAM arrays.
- **Boundary conditions:** identity (all bits pass through unchanged) and full reversal (maximum displacement of every bit) represent the two extremes of the permutation space.
- **Protocol robustness:** back-to-back reconfiguration (TC-05) verifies that `cfg_restart` correctly resets all internal counters and done flags without requiring a full system reset.
- **Non-permutation inputs:** broadcast and dualcast patterns (TC-06) verify that the hardware handles repeated indices in `final_map` gracefully, as may occur during system bring-up or test.

7.4 Formal Verification

Simulation provides functional confidence but cannot exhaustively verify all 1024! possible input permutations. Formal property verification is a natural complement, and two classes of properties are well-suited to formal tools for this design.

7.4.1 FSM Protocol Properties

The configuration FSM has a small, well-defined state space that is directly amenable to bounded model checking or k-induction. The following properties can be expressed as SystemVerilog Assertions (SVA) and verified with tools such as SymbiYosys (open source, Yosys-based) or Cadence JasperGold:

- **Safety:** `cfg_ready` for `cfg_sel=0` deasserts if and only if `done_a` is set, and never reasserts without a `cfg_restart` pulse. Symmetrically for `cfg_sel=1` and `done_b`.
- **Safety:** `ready_out` never asserts unless both `done_a` and `done_b` are set.

- **Safety:** `data_in_vld` is only acted upon when `is_ready` is set; no data word is accepted during configuration.
- **Liveness:** given a continuous stream of valid `cfg_valid` beats with back-pressure respected, `done_a` asserts within exactly `BEATS_TOTAL` cycles.
- **Reset:** after `cfg_restart`, all counters and done flags return to their reset values within one cycle regardless of the current FSM state.

These properties are bounded in depth (at most `BEATS_TOTAL` = 15 cycles to falsify the liveness properties) and do not require abstraction of the data path, making them tractable even without commercial tool licences.

7.4.2 Permutation Composition Correctness

The core correctness property — that `src_for_out[i] == inv_orig_reg[final_req_reg[i]]` for all `i` after the resolving pulse — can be verified formally, but the full 128-bit state space is computationally expensive for current SAT/BDD solvers. Two practical approaches exist:

- **Parameterised proof at reduced width:** prove the property at `DATA_WIDTH=8` or `DATA_WIDTH=16` using bounded model checking. The composition logic is structurally identical at all widths (a generate loop), so a proof at small width provides strong evidence by structural induction. This approach is directly supported by SymbiYosys with the Yosys synthesis backend.
- **Abstraction of the mux tree:** treat `mux_f789_tree` as a black box with a formal specification (`data_out == data_in[select_bin]` after 3 cycles) and verify the crossbar datapath against that spec separately from the mux tree itself. This decouples the state space of the 128-bit data path from the composition proof.

7.4.3 Mux Tree Verification

The `mux_f789_tree` submodule is a strong candidate for standalone formal verification because its correctness property is simple and self-contained: for any valid `select_bin` value in `[0, WIDTH-1]`, `data_out` must equal `data_in[select_bin]` exactly 3 cycles after the inputs are presented. This is a pure combinational equivalence check with a fixed pipeline depth.

Equivalence-checking tools (e.g. Cadence Conformal, Synopsys Formality, or Yosys `equiv_check`) can verify that the MUXF7/F8/F9 tree implementation is logically equivalent to the reference behavioural model:

```
assign data_out_ref = data_in[select_bin]; // behavioural reference
```

The `(* dont_touch *)` attributes on pipeline registers are important here: they prevent Vivado from retiming registers across stage boundaries, ensuring the 3-cycle latency is preserved exactly as specified. Formal equivalence checking after synthesis would confirm that the netlist matches the RTL specification under the timing constraints.

Additionally, the parameter-validation block (\$fatal assertions on WIDTH constraints) can be verified at elaboration time using static analysis, confirming that invalid configurations are caught before simulation or synthesis begins.

7.4.4 Recommended Tool Flow

Property class	Recommended tool	Effort estimate
FSM protocol (safety + liveness)	SymbiYosys + Yosys (open source)	Low — bounded depth, small state space
Composition correctness at reduced width	SymbiYosys k-induction or Cadence JasperGold	Medium — requires parameterised elaboration
Mux tree equivalence	Yosys equiv_check or Synopsys Formality	Low — pure combinational equivalence
Full 128-bit datapath	Cadence JasperGold with abstraction	High — requires manual property decomposition

8. Resource Utilisation

8.1 Post-Implementation Utilisation (DATA_WIDTH=128, integrated end-system)

Resource	Used	Notes
LUT6	46,949 (11.0%)	Dominated by 128 mux_f789_tree instances (128:1 selection per output bit)
MUXF7	17,868 (8.4%)	Second mux stage in mux_f789_tree
MUXF8	8,399 (7.9%)	Third mux stage in mux_f789_tree
FDRE (flip-flops)	33,086 (3.9%)	Pipeline registers: broadcast tree, mux stages, output, configuration state
LUTRAM	195	inv_orig_reg + final_req_reg (2 × 128 × 7b)
BRAM	0	Not used; configuration arrays require parallel async read

The ZCU111 (XCZU28DR) provides 425,280 LUTs and 850,560 flip-flops. The integrated 128-bit end-system — crossbar, AXI-Stream wrapper, Aurora core, and Zynq PS interconnect — consumes approximately 11% of LUTs and 4% of flip-flops, leaving the large majority of the fabric free for decoder accelerator blocks. These are measured post-route figures for the full integrated design, not estimates.

9. Key Design Decisions

9.1 Why LUTRAM Instead of BRAM for Configuration Storage

The composition step requires reading all 128 entries of `inv_orig_reg` and `final_req_reg` simultaneously (one read per output bit, all in the same clock cycle). BRAM supports at most two read ports; LUTRAM provides fully asynchronous parallel read with no port limit. The 1,792-bit arrays fit comfortably in LUTRAM at approximately 320 LUT primitives each.

9.2 Why Two Maps Instead of One

Sending a single pre-composed permutation vector would require the upstream system to compute the composition in software before transmission. The two-map representation delegates this computation to the FPGA, reduces the semantic burden on the sender, and allows the maps to be loaded independently and asynchronously (`cfg_sel=0` and `cfg_sel=1` are independently back-pressured). The hardware composition adds only ~ 3 cycles of reconfiguration latency.

9.3 Multicycle Path Constraints vs. RTL Restructuring

The configuration-phase write paths (particularly the data-dependent write address of `inv_orig_reg`) create logic depths of 10 LUT levels that violate the 2.5 ns clock period when the module is integrated in a design with a high-fanout PS clock tree. Two approaches were considered:

- **RTL restructuring:** split the write into two pipeline stages. This fixes the standalone timing but introduces a race condition where the resolving signal can fire before the last pipelined write has settled, requiring careful multi-cycle done logic. Multiple iterations were required to eliminate the race.
- **Multicycle path constraints:** declare the three configuration-phase paths as 2-cycle paths in the XDC. This is functionally safe because these paths are only active during `S_LOAD`, never during `S_READY` (data processing). The RTL remains clean and the constraint is self-documenting.

The multicycle constraint approach was adopted for the integrated design as it is both simpler and more robust. The constraints are annotated in `constraints.xdc` with the rationale for each.

9.4 Broadcast Tree for the Data Input

Driving the full data-input word to every mux instance creates significant routing pressure if the signal is not replicated. The 32-cluster broadcast tree registers `data_in` into 32 independent copies, each driving 32 mux instances. This limits the fanout of each `data_in` bit to 32, improving routing and allowing Vivado to

place each cluster close to its mux group. The (* dont_touch *) attribute prevents Vivado from collapsing the replication.

10. Build and Simulation Instructions

10.1 Repository Structure

The project is organised into three independently buildable stages, each containing its own Vivado build script, RTL sources, testbench and constraints:

Directory	Contents	Output
crossbar/	runtime_configurable_crossbar.sv, mux_f789_tree.sv, tb_runtime_configurable_crossbar.sv, constraints.xdc, prj.tcl	Synthesised crossbar IP core; post-synthesis timing report
axis_wrapper/ /	crossbar_axis_wrapper.sv, tb_crossbar_axis_wrapper.sv, prj.tcl	Synthesised AXI-Stream wrapper IP core; end-to-end latency report
end_system/	bd.tcl, aurora_zcu111.xdc, constraints.xdc	Full Aurora + crossbar + Zynq PS block design; implementation bitstream

10.2 Build Automation

A top-level Makefile and shell script automate the complete build sequence. The three stages are built in order: crossbar IP first (it must be packaged before the wrapper can reference it), then the wrapper IP, then the end-system block design.

To build all three stages in sequence:

```
make
# or, equivalently:
make build
```

To clean all generated outputs (Vivado project directories, log files, IP caches):

```
make clean
```

The `vivado_build_all.sh` script iterates over the three subdirectories and calls each stage's own `vivado_build.sh`. It exits immediately on any non-zero return code so that a failed synthesis or DRC in an earlier stage does not silently propagate to later stages.

10.3 Per-Stage Build

Each stage can also be built independently. From any stage directory:

```
vivado -mode batch -source prj.tcl
```


The `prj.tcl` script creates the Vivado project, adds sources, applies constraints, launches synthesis with 6 parallel jobs, opens the synthesised design, and writes post-synthesis timing and utilisation reports one level above the project directory for easy access. For the crossbar stage, `prj.tcl` also sets the `AltSpreadLogic_high` placement directive, `AggressiveExplore` routing directive, and a `Pblock` floorplan constraint to mitigate routing congestion — see Section 6.3 for rationale.

10.4 File List

File	Description
Makefile	Top-level build entry point: <code>make</code> / <code>make build</code> / <code>make clean</code>
<code>vivado_build_all.sh</code>	Orchestrates sequential build across crossbar, <code>axis_wrapper</code> , <code>end_system</code>
<code>clean.sh</code>	Removes all generated Vivado project files and outputs
<code>crossbar/ runtime_configurable_crossbar.sv</code>	Crossbar top-level RTL
<code>crossbar/mux_f789_tree.sv</code>	Pipelined parameterised mux submodule (128:1 in this build)
<code>crossbar/ tb_runtime_configurable_crossbar.sv</code>	Crossbar self-checking testbench (7 test cases)
<code>crossbar/constraints.xdc</code>	400 MHz clock + multicycle path constraints (standalone, no hierarchy prefix)
<code>crossbar/ constraints_standalone.xdc</code>	Same constraints with unqualified cell names for standalone implementation
<code>crossbar/prj.tcl</code>	Vivado project, synthesis, implementation (with congestion directives) and IP packaging
<code>axis_wrapper/ crossbar_axis_wrapper.sv</code>	AXI4-Stream wrapper RTL
<code>axis_wrapper/ tb_crossbar_axis_wrapper.sv</code>	AXI-Stream wrapper testbench
<code>end_system/bd.tcl</code>	Vivado IPI block design (Aurora + crossbar + Zynq PS)
<code>end_system/aurora_zcu111.xdc</code>	ZCU111 GTY pin assignments and

File	Description
	Aurora clock constraints
end_system/constraints.xdc	Multicycle path constraints with full Aurora block design hierarchy paths

10.5 Simulation

Simulation can be run from the Vivado GUI after project creation (Run Simulation → Run Behavioral Simulation) or via the Tcl console:

```
launch_simulation
run all
```

Expected output for a passing crossbar testbench run:

```
=====
SIMULATION COMPLETE   PASS:188   FAIL:0
=====
```

11. Alternative Architectures Evaluated

Before settling on the pipelined MUXF7/F8/F9 tree approach, three alternative multiplexer implementations were investigated. All three are documented below for completeness.

11.1 CARRY8-Based One-Hot Multiplexer

The first alternative implements each 1024:1 mux using a cascaded CARRY8 chain. The binary select address is first decoded into a 1024-bit one-hot vector. Each CARRY8 primitive then performs a single AND between one data bit and its corresponding one-hot bit, propagating the result through the carry chain. The final carry-out is the selected data bit.

The theoretical appeal is that CARRY8 uses dedicated high-speed vertical routing between CLBs, avoiding the general routing fabric. However, a cascaded chain of 128 CARRY8 primitives (1024 bits / 8 bits per primitive) creates a long ripple path that is difficult to pipeline and whose timing degrades significantly with routing congestion in a full 1024-output implementation.

The one-hot decode logic itself also adds combinational depth before the carry chain. In practice the CARRY8 approach performs well for narrower muxes but does not scale cleanly to 1024 inputs at 400 MHz.

11.2 Batchier Bitonic Sorting Network

The second alternative reframes permutation as a sorting problem. Each input bit i is packed with its destination index into an 11-bit packet $\{\text{dest}[i][9:0], \text{data_in}[i]\}$. A Batchier Bitonic Sorting Network then sorts all 1024 packets by destination tag. After sorting, the packet at position k carries the data bit that belongs at output k .

For $N=1024 = 2^{10}$, the Batchier construction requires $P=10$ passes with $P(P+1)/2 = 55$ total compare-swap stages, each containing 512 independent compare-swap cells. The network is fully pipelined with one register stage per step, giving a total latency of 56 cycles.

Metric	MUXF7/F8/F9 Tree (chosen)	Bitonic Sorter
Pipeline latency	4 cycles	56 cycles
Throughput	1 word / cycle	1 word / cycle
Pipeline flip-flops	~6K	~580K ($55 \times 1024 \times 11$ bits)
Configuration maps	2 (orig + final)	1 (dest only)
Configuration beats	342 (171×2)	171

Metric	MUXF7/F8/F9 Tree (chosen)	Bitonic Sorter
Inverse-map logic	Yes (in loader)	Not required
Critical path	3 LUT levels (MUXF7/8/9)	Compare-swap cell + routing
Timing at 400 MHz	Met (+1.3 ns WNS)	Requires careful pipelining

11.3 Benes Permutation Network

A Benes network is a classical rearrangeable non-blocking switching fabric originally proposed for telephone routing. For N inputs, it consists of $2 \cdot \log_2(N) - 1$ stages of $N/2$ independent 2×2 switches, giving 19 stages for $N=1024$. Every possible permutation can be realised by setting the switch controls appropriately. The network uses $O(N \log N)$ switches compared to $O(N^2)$ for a full crossbar, making it area-efficient for large N .

For FPGA implementation each 2×2 switch is trivially a single 2:1 mux — one LUT per data bit. With 19 stages and one pipeline register per stage, a Benes network achieves 19 cycles of latency. The switch-control bits ($19 \times 512 = 9,728$ bits total) must be pre-computed from the desired permutation using a recursive routing algorithm (Waksman/looping algorithm), which is non-trivial to implement in hardware at runtime.

The key difficulty with Benes networks in a runtime-reconfigurable context is the control generation. Computing the switch settings from an arbitrary permutation requires a recursive decomposition algorithm whose hardware implementation adds significant complexity and latency on top of the 19-cycle data pipeline. For static or slowly-changing permutations this is acceptable, but for the QEC use case where permutations may change frequently, the control computation overhead is a concern.

Metric	MUXF7/F8/F9 Tree (chosen)	Benes Network
Pipeline stages	3 (data path)	19 ($2 \cdot \log_2(1024) - 1$)
Pipeline latency	4 cycles total	19+ cycles
Switch/mux count	1024 mux instances	$9,728 \times 2 \times 2$ switches
Control bits	10-bit select per output (pre-computed $P[i]$)	9,728 switch bits (recursive algorithm)
Control generation	Simple composition (2 LUTRAM lookups)	Waksman looping algorithm — complex hardware

Metric	MUXF7/F8/F9 Tree (chosen)	Benes Network
FPGA primitives	Dedicated MUXF7/F8/F9 silicon	General LUTs
Routing congestion	Moderate (cluster replication mitigates)	Lower ($O(N \log N)$ wires vs $O(N^2)$)

11.4 Permutation Networks in QEC Hardware

In published QEC FPGA implementations, bit-level permutation is typically handled implicitly through fixed wiring rather than a dedicated programmable crossbar. The syndrome bits arrive with a known fixed mapping from the qubit readout hardware and the decoder is designed to match that layout. Runtime reprogrammability of the permutation is not a feature of most existing decoders.

Recent QEC FPGA decoders — including the Helios distributed Union-Find decoder (11.5 ns per measurement round for $d = 21$ on a single VCU129 [4]), its multi-FPGA extension DecoNet (100 logical qubits with lattice surgery at 2.40 μ s average latency [5]), and recent FPGA neural-network decoders demonstrating sub-microsecond closed-loop QEC on real superconducting hardware [6] — all implement fixed syndrome-to-decoder mappings determined at build time. The interconnect latency budget across these systems is typically a dominant constraint, leaving little room for software-based pre-processing of the syndrome stream.

The motivation for a runtime-configurable permutation arises specifically when the same hardware platform must support multiple qubit connectivity graphs, different QEC codes, or different logical-qubit encodings without bitstream recompilation — precisely the scenario in a multi-tenant or rapidly evolving QEC stack. In this context, the ~ 83 ns reconfiguration latency of this design is acceptable: it is fast enough to reconfigure between QEC rounds or between different logical-qubit operations, while the 4-cycle (10 ns at 400 MHz) data-path latency adds negligible overhead to the decode loop.

11.5 Why MUXF7/F8/F9 Was Selected

The Xilinx UltraScale+ CLB contains dedicated MUXF7, MUXF8 and MUXF9 primitives that implement wide multiplexers in silicon, outside the general LUT fabric. A single CLB can implement a 16:1 mux in one stage using LUT6 + MUXF7 + MUXF8. For a 128:1 selection this decomposes into two pipelined stages — a 16:1 first stage consuming select[3:0], followed by an 8:1 second stage consuming select[6:4] — each with a combinational delay of approximately 0.5 ns.

Compared to the alternatives: the CARRY8 ripple chain is eliminated because it creates a long combinational path that is difficult to pipeline at 400 MHz. The bitonic sorter is eliminated because its 56-cycle latency is $14\times$ higher than the

mux tree, and its $\sim 97\times$ higher flip-flop count ($\sim 580\text{K}$ vs $\sim 6\text{K}$) reduces the resources available for decoder accelerators on the same device. The Benes network offers better area efficiency than the full mux tree but requires a complex recursive control-generation algorithm that adds hardware complexity and reconfiguration overhead for a runtime-configurable design.

The MUXF7/F8/F9 tree wins on all the metrics that matter for the QEC use case: lowest data-path latency (4 cycles), simplest control generation (two LUTRAM lookups per output bit, computed in a single composition cycle), timing closure at 400 MHz with positive slack, and direct mapping to dedicated silicon paths that Vivado can optimally place and route.

13. Implementation Findings — Achievable Permutation Width

Sections 6 and 8 report timing and resource figures derived from synthesis of the 1024-bit configuration. This section records the results of the full place-and-route and bitstream-generation campaign for the integrated end-system, which establish the permutation width that is actually achievable on the ZCU111 once the crossbar shares the device with the Aurora link, the Zynq processing system, and the AXI infrastructure. The headline result is that the deliverable integrated design uses a DATA_WIDTH of 128, not 1024: the larger widths are limited not by logic capacity but by routing congestion, which scales far more steeply with permutation width than LUT count alone suggests.

13.1 Width Sweep Results

The crossbar was built at several permutation widths, both standalone (out-of-context) and integrated in the full Aurora block design. The standalone and integrated results differ markedly because the integrated design must route the crossbar through the same interconnect resources used by the processing system and the Aurora core. The table below summarises the outcome at each width.

DATA_WIDTH	Standalone (OOC)	Integrated end-system	Limiting factor
128	Fits, timing met	Routes clean, DRC clean, bitstream generated	None — supported configuration
256	Fits (OOC, whole die)	Unroutable — global congestion level 6	Interconnect congestion, not logic
1024	Does not place	Not attempted	~7x logic over-utilisation (est. 3.1M LUT6 vs 425k available)

The 1024-bit configuration referenced elsewhere in this document is a synthesis-stage estimate. It does not place on the XCZU28DR: the flat NxN multiplexer fabric scales as $O(N^2)$ in logic, and at $N=1024$ the mux tree requires on the order of seven times the available LUT capacity of the device. The 256-bit configuration places but cannot be routed in the integrated system; the 128-bit configuration is the largest flat crossbar that completes place-and-route and bitstream generation.

13.2 Integrated Design — 128-bit Implementation Results

The 128-bit integrated design completes the full implementation flow. Post-route results are summarised below. Utilisation is comfortable and routing is clean; the design is not resource-bound, which confirms that the limit on permutation width is interconnect routing rather than fabric capacity.

Metric	Value (128-bit integrated design)
CLB LUTs	46,949 of 425,280 (11.0%)
CLB Registers	33,086 of 850,560 (3.9%)
F7 Muxes	17,868 of 212,640 (8.4%)
F8 Muxes	8,399 of 106,320 (7.9%)
Block RAM	3.5 of 1,080 (0.3%)
DSP	0 of 4,272
Routing status	Fully routed, 0 nets with routing errors
DRC	0 errors (7 warnings, all benign — see 13.4)

13.3 Why 256-bit Does Not Route in the Integrated System

Standalone, the crossbar fits and meets timing at widths well beyond 128, because an out-of-context build gives the mux fabric the entire device with no competing logic and no I/O. This standalone result does not predict the integrated outcome. In the end-system the crossbar shares the die and, critically, the routing channels with the Zynq processing system, the Aurora 64B/66B core, the AXI interconnect, and two asynchronous FIFOs.

At DATA_WIDTH=256 the router reports global congestion level 6 — the maximum on Vivado's 0–8 scale, where level 5 and above indicates the router is likely to fail — with directional congestion of 86–97% across all four directions. The error [Route 35-3] “Design is not routable” was observed on three separate implementation attempts. A floorplan pblock confining the crossbar, combined with the AltSpreadLogic_high placement directive and the AggressiveExplore routing directive, was applied as a mitigation. It did not resolve the congestion: floorplanning and congestion-aware directives typically recover on the order of 10–20% of routing headroom, and the 256-bit deficit is substantially larger than that. The conclusion is that a flat 256-bit permutation crossbar exceeds the routable interconnect capacity of the XCZU28DR once the Aurora and processing-system subsystems are present, and that this is an architectural limit rather than a tuning problem.

13.4 Residual Implementation Warnings

The 128-bit integrated build produces seven DRC warnings and zero DRC errors. Six are PDCN-1569 “LUT equation term check” warnings, all located inside the Xilinx-generated AXI data-width converter FIFOs in the processing-system interconnect; they are a known benign artefact of the FIFO generator's LUT mapping and are not related to the crossbar. The seventh is an RTSTAT-10 “no routable loads” warning covering 106 nets, most in processing-system FIFO reset logic; two of these nets belong to the AXI-Stream wrapper's second skid-buffer stage (the tkeep and tlast outputs), whose downstream loads were trimmed because the consuming logic regenerates those signals rather than propagating them. None of the seven warnings blocks bitstream generation, and all are benign for the QEC data path, in which tkeep is held constant and tlast is regenerated by beat count.

13.5 Implications for Larger Permutation Widths

If an application requires a permutation wider than 128 elements, the flat MUXF7/F8/F9 crossbar described in this document cannot provide it within the integrated ZCU111 system, for the routing reasons established above. Two architectural routes extend the achievable width, both requiring a redesign of the crossbar core rather than a parameter change.

The first is a multi-stage rearrangeable network — a Beneš network, evaluated in Section 11.3. Its routing complexity is $O(N \log N)$ rather than $O(N^2)$, which is the property that makes wider permutations routable; the data-path latency of 13–17 pipeline stages remains comfortably within a 1 μ s QEC round. The cost is the recursive switch-setting computation noted in Section 11.3, which is significant only if the permutation changes on every round rather than per experiment.

The second is time-multiplexing: reusing a single physical 128-bit crossbar over multiple passes to realise a wider permutation. This reuses the proven 128-bit core and keeps the routing footprint at the known-good 128-bit figure, at the cost of proportionally higher latency, a sequencing controller, and increased configuration time. In the general case an arbitrary 256-element permutation requires four passes of a 128-bit crossbar, since any output may draw from either input half. Time-multiplexing is attractive when the permutation is static or slow-changing and when syndromes arrive with sufficient inter-arrival gap; for permutations that change every round, the repeated configuration-load cost erodes the latency budget the approach is intended to protect.

For the present deliverable, the 128-bit flat crossbar is the supported configuration. It is sufficient for surface-code syndromes up to approximately distance 11 when one permutation element is allocated per stabiliser bit; applications requiring wider syndromes, multi-round decoding windows, or per-element soft information would consume the 128 elements faster and should adopt one of the two architectural routes above.

13.6 Scaling Path to 1024-Element Permutations

The RTL is fully width-generic: DATA_WIDTH and ADDR_WIDTH are parameters, the configuration loader, composition logic, and mux tree are all generate-loop constructs, and the design has been elaborated and simulated at widths from 32 up to 1024 bits. Width is therefore not limited by the source: it is limited by what a given device can place and route. It is important to separate the two ceilings the build campaign established, because they have different causes and different remedies.

The first ceiling is logic capacity. The flat MUXF7/F8/F9 crossbar is an N-to-1 multiplexer replicated N times, so its primitive count grows as $O(N^2)$. On the XCZU28DR this exhausts the LUT fabric between 256 and 1024 bits — the 1024-bit configuration synthesises but does not place, requiring on the order of seven times the device's LUT capacity. A larger device with proportionally more CLB columns relaxes this ceiling, and the largest UltraScale+ and Versal parts have several times the logic of the XCZU28DR. The second ceiling is routing congestion, and it is the harder one: the all-to-all wiring of a flat crossbar means interconnect demand also grows as $O(N^2)$, and congestion grows faster than device size does. This is why a 256-bit flat crossbar is already unroutable in the integrated system on this device (Section 13.3). A bigger FPGA does not, on its own, make a flat 1024-bit crossbar routable — the flat architecture is the limiting factor, not the part.

Reaching a 1024-element permutation is therefore a realistic target, but only with the multi-stage network architecture identified in Section 11.3 and Section 13.5. A Beneš network performs the same arbitrary permutation with $O(N \log N)$ switches and $O(N \log N)$ interconnect rather than $O(N^2)$. For $N=1024$ this is $2 \cdot \log_2(1024) - 1 = 19$ switch stages of 512 two-input switches each — on the order of 9,700 two-input multiplexers in total, compared with the order of a million LUTs a flat 1024-bit crossbar would require. That is a reduction of roughly two orders of magnitude in both logic and routing demand, which is what brings a 1024-element permutation within reach of a large UltraScale+ or Versal device. The data-path latency of 13–19 pipeline stages remains well inside a 1 μ s QEC round.

The table below summarises the scaling picture. The flat-crossbar column reflects what this project measured directly; the Beneš column is the projected path, and would require the core redesign and the switch-setting computation described in Section 11.3 before it could be claimed as a measured result.

Permutation width	Flat crossbar (this design)	Beneš network (projected)	Suitable device
128	Routes in integrated ZCU111 system	Routes comfortably	XCZU28DR (ZCU111)

256	Unroutable (congestion level 6)	Routes; ~15 switch stages	XCZU28DR or larger
512	Does not place	Routes; ~17 switch stages	Larger UltraScale+ / Versal
1024	Does not place (~7x over capacity)	Projected to route; 19 switch stages	Large UltraScale+ / Versal

In summary, the design is parameterised for any power-of-two width up to 1024, and a 1024-element permutation is achievable in principle; but it is achievable through the Beneš architecture on a larger device, not by recompiling the flat crossbar of this deliverable for a bigger part. The flat 128-bit crossbar is what has been built, routed, and verified on the ZCU111; the 1024-element capability is a documented and credible extension, contingent on the architectural work identified above.

12. Conclusions and Future Work

12.1 Summary

A runtime-configurable bit-permutation crossbar has been designed, implemented and verified for deployment on the Xilinx ZCU111 RFSoc platform. The component fills a specific gap in the published QEC FPGA decoder landscape: a runtime-reprogrammable bit-routing layer between the fixed hardware ordering of syndrome bits and the logical ordering expected by the decoder. The key deliverables are summarised below.

- **Correctness:** functional correctness demonstrated across identity, reversal, random, back-to-back reconfiguration and broadcast/dualcast test cases with bit-exact golden-model comparison (188 vectors, 0 failures).
- **Speed:** reconfiguration latency ~ 83 ns at 400 MHz, comfortably under the 1 μ s budget. Data-path latency of 4 clock cycles (10 ns) and sustained throughput of one 128-bit word per cycle.
- **Quality of code:** fully parameterised SystemVerilog with inline architectural rationale, structured testbench, and a build flow split into three independently buildable stages.
- **Quality of testing:** self-checking simulation framework with quantitative latency measurement, plus a documented plan for formal property verification with concrete tool recommendations.
- **Quality of documentation:** this document, accompanied by inline comments in every RTL file and per-stage README content in the build scripts.
- **Integration:** clean integration with an Aurora 64B/66B serial link via AXI4-Stream, forming a complete receive-permute-transmit data path on the ZCU111. The component can therefore be exercised between two physical FPGAs over a single SFP28 link.

12.2 Potential Extensions

- **Streaming reconfiguration:** modify the wrapper to accept back-to-back 32-beat packets, re-entering RX_CFG_A after RX_IDLE, to support continuous permutation updates at line rate.
- **Dual-port configuration:** add a second cfg_data bus so orig_map and final_map can be loaded in parallel, halving reconfiguration latency to approximately 430 ns.
- **Sub-1024 widths:** the DATA_WIDTH and ADDR_WIDTH parameters are fully generic; the design has been validated at DATA_WIDTH=128 and DATA_WIDTH=32 during development.
- **ASIC portability:** removing the (* ram_style *) and (* dont_touch *) Xilinx-specific attributes and replacing mux_f789_tree with a technology-agnostic tree mux would make the design suitable for ASIC implementation.

- **Formal verification:** execute the proposed SymbiYosys/JasperGold proof plan from Section 7.4, providing exhaustive correctness guarantees beyond what simulation can offer.

Appendix A — Glossary

Term	Definition
AXI4-Stream (AXI-S)	ARM AMBA streaming interface protocol; tdata payload with tvalid/tready handshake and optional tlast packet boundary.
Aurora 64B/66B	Xilinx-provided low-overhead serial link IP, used here at 25.78 Gbps over SFP28.
BRAM	Block RAM; hardened FPGA memory primitive with up to two ports.
CARRY8	Xilinx UltraScale+ carry-chain primitive (8-bit), used for fast arithmetic and chained logic.
CLB	Configurable Logic Block; the basic Xilinx fabric tile containing LUTs, FFs and MUXF primitives.
FF / FDRE	Flip-flop / D flip-flop with clock-enable and synchronous reset.
GTY	Xilinx high-speed serial transceiver family supporting up to 33 Gbps per lane.
LUT / LUT6	Look-Up Table; the basic combinational primitive (6 inputs in UltraScale+).
LUTRAM	Distributed RAM built from LUTs; supports asynchronous parallel read.
MUXF7/F8/F9	Hardened wide-mux primitives in UltraScale+ CLBs, enabling 16:1 / 32:1 / 64:1 selection with minimal logic depth.
pl_clk0	PS-to-PL clock 0 from the Zynq UltraScale+ processing system; used here at 400 MHz.
QEC	Quantum Error Correction; protocols that protect logical qubits from physical errors via repeated syndrome measurement and decoding.
RFSoc	Zynq UltraScale+ RFSoc: SoC variant integrating RF data converters, GTY transceivers, programmable logic and an Arm processing system.
SFP28	Small Form-factor Pluggable transceiver cage supporting 25 Gbps optical/copper modules.
Syndrome	Output of a stabiliser measurement in a QEC code; the

Term	Definition
	data the decoder consumes.
WNS	Worst Negative Slack: timing-analysis metric; positive means the timing constraint is met with margin.